

AI Bot Director Plan for a WoW Private Server

Neural-net assisted bot LOD, economy control, dungeon support, personality, and distributed scaling

Prepared as a reusable planning report based on the architecture discussion

1. Executive Summary

The goal is not to run 5,000-10,000 fully simulated player bots at all times. The goal is to create 5,000-10,000 persistent bot identities and make the world feel alive while only fully simulating the bots that are visible, grouped, fighting, trading, or otherwise relevant to real players. The system should combine traditional game AI with neural networks and an AI director service.

- Use deterministic server rules for correctness: spells, auras, combat, quest validation, inventory, auction creation, and database writes.
- Use neural networks for high-level decisions: bot skill/personality, zone population balancing, economy health, dungeon role filling, path style selection, and player-experience tuning.
- Use the GPU for batch inference and language/chat services, not for core server tick logic such as aura ticking or DB-heavy operations.
- Use simulation LOD so far-away bots progress statistically instead of pathing, fighting, looting, and casting in full detail.
- If one machine is not enough, distribute planning and abstract simulation to worker nodes while keeping the WoW server as the authoritative source of truth.

2. Target Architecture

Recommended high-level architecture:

Component	Responsibility
Worldserver / Mangos Core	Authoritative gameplay state: movement legality, combat, auras, quest status, inventory, auction house, packets, persistence.
BotBridge Module	Safe API between the server and AI services. Exposes validated commands such as promote bot, demote bot, queue dungeon bot, abstract progress, post auction.
AI Director Service	Global brain that observes server health, player distribution, economy, dungeon demand, and bot identity states. It decides what needs adjustment.
Bot LOD Manager	Chooses whether each bot is full simulation, light

	simulation, abstract simulation, or offline/progress-only.
Economy Director	Controls gathering, crafting, auctions, item supply, gold faucets, prices, and scarcity targets.
Dungeon / Group Director	Maintains role availability by promoting suitable bots as tanks, healers, or DPS for any level bracket.
Personality + Skill Model	Assigns bots unique playstyle, skill tier, APM, reaction time, risk tolerance, leveling preference, PvP aggression, and social behavior.
GPU Inference / LLM Service	Handles batched NN inference, bot dialogue, player intent understanding, and style/personality text generation.
Worker Nodes	Optional distributed services for abstract simulation, economy planning, dungeon matching, chat generation, and behavior scoring.

3. Simulation LOD: The Main Scaling Tool

The system should treat bots like graphics LOD. Only nearby or important bots need full simulation. Far-away bots should be simulated as outcomes, not detailed actions.

Mode	When Used	Update Rate	Behavior
LOD 0: Full Bot	Visible to real players; grouped; dungeon; PvP; fighting player; trading/AH interaction	Normal server update	Runs real movement, combat, targeting, spells, packets, loot, quest logic.
LOD 1: Light Bot	Near a player but not visible or not directly relevant	1-5 seconds	Low-frequency movement and state updates. Limited scanning and simplified behavior.
LOD 2: Abstract Bot	Same zone/map but not seen; doing normal leveling, gathering, or travel	30-120 seconds	No real pathing or combat. Uses formulas for XP, loot, gold, quest progress, travel, gathering.
LOD 3: Offline / Progress Only	No player nearby; bot identity should still develop	5-15 minutes	Character progresses by scheduled profiles. No spawn, packets, movement, or world object updates.

4. What Neural Networks Should Control

Neural networks should influence high-level choices and style, while deterministic game code executes legal actions. This keeps the server stable and prevents the AI from corrupting game state.

- AI director decisions: how many bots should be active, where bots should go, what resources should be farmed, how dungeon roles should be filled, and when bots should be promoted/demoted.
- Bot skill and personality: reaction time, APM, interrupt chance, target swapping, risk tolerance, PvP aggression, helping behavior, leveling route preference, gathering preference, and social behavior.
- Human-like pathing style: imperfect routes, pauses, jitter, camera/mouse-like facing changes, jumping, bunny hopping, path variation, and occasional inefficient behavior.
- Economy tuning: item supply, auction pricing, scarcity, gold faucet limits, gathering allocation, crafting output, and sell-through predictions.
- Player experience prediction: whether the world feels crowded, empty, too hard, too easy, economically broken, or under-supported for group content.

5. What Should Stay Deterministic

These systems should remain in the server or traditional bot code. NN can suggest intentions, but the server should validate and execute them.

- Spell execution, cooldowns, procs, aura ticking, resist/miss/dodge/parry logic, threat, and combat event ordering.
- Inventory, quest credit, loot generation, auction creation, mail, trading, and database writes.
- Line of sight and visibility rules that directly affect fairness or packet delivery.
- Low-level pathfinding correctness and collision legality. NN can choose style overlays, not illegal movement.
- Security and validation. External AI services should never directly mutate character tables or auction tables.

6. NN-Enhanced Human-Like Pathing

The best approach is not to have a neural net output raw movement keys every tick. Instead, use normal pathfinding to produce a valid path, then apply a humanization layer that is partly rule-based and partly NN-driven.

- Base path: normal pathfinder produces legal waypoints.
- Style model: NN selects a movement style profile based on bot personality, skill tier, location, player proximity, and current activity.
- Humanization overlay: adds small deviations, pauses, facing changes, strafes, jump events, bunny hops, route preference, and occasional inefficient behavior.
- Visibility-aware behavior: when a real player can see the bot, increase human-like detail. When unseen, use minimal movement or abstract travel.
- Safety rules: never let the humanizer create wall clipping, impossible slopes, teleport-looking movement near players, or invalid paths.

Example movement style outputs:

Style	Typical Use	Parameters	Effect
Focused	Dungeon, combat approach, travel with purpose	Low pause, low jitter, direct routing, fast facing	Looks efficient and experienced.
Casual	Leveling, town movement, questing	Moderate pauses, small path jitter, occasional jumps	Looks like a normal player wandering.
Noob	Low-skill bot, starter zones	More hesitation, route mistakes, delayed reactions, odd facing	Feels less robotic and easier to beat.
PvP Pro	Battleground/world PvP	Strafes, fake retreats, jump turns, line-of-sight dancing	Higher pressure and more player-like movement.
Gatherer	Herb/mining routes	Node-focused routing, small detours, frequent stops	Looks like a farming character.

7. Bot Skill Tiers and Personality Models

Each bot identity should have a stable personality and a skill tier. This gives variety and makes PvP and world interactions feel less repetitive.

Tier	Reaction Time	APM	Combat Behavior	Mistakes
Easy	900-1600 ms	Low	Basic rotation, poor target swaps, rarely interrupts	Frequent inefficient choices, overpulls, delayed heals.
Normal	500-1000 ms	Moderate	Uses decent rotation, some interrupts, basic positioning	Occasional poor cooldowns or late reactions.
Hard	250-600 ms	High	Good rotations, better target priority, defensive cooldowns	Fewer mistakes, adapts to player behavior.
Pro	100-350 ms	Very high but capped	Strong PvP movement, interrupts, LOS usage, coordinated cooldowns	Rare mistakes, but intentionally human-limited.
Roleplay / Casual	Variable	Low-moderate	May prioritize social, questing, gathering, or exploration over	Makes flavor-driven choices.

Suggested personality dimensions:

- Aggression vs caution
- Efficiency vs roleplay
- Solo preference vs group preference
- Economy focus vs leveling focus
- PvP hunger vs PvP avoidance
- Helper tendency toward real players
- Risk tolerance for elite quests or contested zones
- Chatty vs quiet
- Noob-like vs optimized gameplay

8. Learning From Bot Gameplay Over Time

The system can collect telemetry from bot actions and use it to improve difficulty models, economy tuning, and pathing style. This should start as logging and offline training, not live uncontrolled self-modification.

- Log combat outcomes: win/loss, damage done, damage taken, interrupts, dispels, reaction delays, deaths, escape success, potion/cooldown usage.
- Log PvP encounters: opponent level/class, time to death, target swaps, LOS use, CC chains, distance management, player complaints or positive outcomes.
- Log pathing metrics: route length vs optimal, stuck events, jumps, pauses, facing changes, player-visible movement weirdness.
- Log economy outcomes: auction sell-through, price volatility, material shortages, gold inflation, farming throughput, item sink/faucet balance.
- Train models offline first; then deploy versioned models such as movement_v3, economy_v2, pvp_hard_v4. Keep rollback ability.

9. GPU Utilization Plan

The RTX GPU should be used where it adds real value: batched model inference and language generation. Core world simulation should stay CPU-side unless the engine is heavily redesigned.

Good GPU Use	Why	Example
Batch bot intent inference	Same model applied to thousands of compact bot state vectors	Every 1-10 seconds, score 5k bots for next high-level intent.
LLM bot speech	Language generation is GPU-friendly and expensive on CPU	Generate whispers, party chat, RP lines, dungeon responses.
Player intent understanding	Classify player chat or commands into useful bot actions	Player says: need heals for SM. System detects dungeon + role need.
Economy prediction	Numeric modeling over item	Predict shortage of copper ore or

	supply/demand and pricing	low-level bags.
Difficulty scoring	Evaluate combat telemetry and adjust bot tier profiles	Tune Easy/Normal/Hard/Pro model parameters.

Poor GPU targets: pathfinding per bot, combat spell execution, aura ticking, packet-level movement updates, and database-heavy work. These are branchy, stateful, latency-sensitive, and tied to live server memory.

10. Economy Director

The economy director should make bots gather, craft, buy, sell, and consume in a way that keeps the auction house alive without flooding or inflating the economy.

- Track supply, demand, median price, sell-through rate, item age, level-band demand, crafting dependencies, gold generation, and item sinks.
- Set quotas for raw gold, rare items, gathering materials, crafted goods, and dungeon drops. Avoid infinite item creation.
- Assign abstract gatherer/crafter bots to shortages: mining, herbalism, skinning, tailoring, bags, potions, food, reagents, enchants.
- Use bot auctions to fill market gaps, not to dominate every market. Keep believable prices and random listing behavior.
- Demote or pause farming profiles when supply is too high or prices collapse.

11. Dungeon and Role Availability

A dungeon director should ensure that players can find tanks, healers, and DPS across level brackets without requiring thousands of full-time active bots.

- Maintain pools of suitable bot identities by level, class, role, gear, and skill tier.
- When a player needs a dungeon group, promote matching bots from abstract/offline mode into full mode.
- Teleport or route bots plausibly depending on visibility and immersion settings. Never visibly teleport in front of players unless acceptable for your server style.
- After the dungeon, apply real rewards if simulated fully; then demote bots back to light/abstract/offline modes.
- Use skill/personality to vary dungeon experience: cautious tank, aggressive rogue, slow healer, pro mage, chatty roleplayer.

12. Distributed Bot Driver Scaling

If one machine cannot handle the desired number of bots, split planning and abstract simulation across networked worker machines. Keep the game server authoritative.

- Do not start by running thousands of full external client connections. That tests networking but does not solve server-side AI CPU cost.

- External workers should compute intentions and abstract outcomes, not directly write DB state.
- Use queues or RPC for commands: promote bot, demote bot, assign gathering profile, request dungeon fill, generate chat, post auction request.
- Partition work by responsibility: economy worker, chat worker, dungeon worker, abstract leveling worker, movement/personality scoring worker.
- Use idempotent commands and versioned state snapshots so workers can recover if disconnected.

13. Implementation Roadmap

Phase	Goal	Build	Success Metric
1	Measure current bottlenecks	Metrics for update time, pathfinding, visibility, combat, bot count, DB load	Know exactly what breaks at 2k bots.
2	Add bot LOD modes	Full/light/abstract/offline state machine and update throttles	Far bots stop consuming full CPU.
3	Abstract progress engine	XP/gold/loot/quest/gathering formulas and profiles	Bots level and gather while unseen.
4	Rule-based director	Zone density, dungeon role fill, economy quotas	World feels alive without NN yet.
5	Human-like movement overlay	Jitter, pauses, jumps, facing, route variation, stuck safeguards	Visible bots stop looking like straight-line robots.
6	Skill/personality profiles	Easy/Normal/Hard/Pro, APM, reaction time, combat style	PvP and dungeons feel varied.
7	Telemetry + offline training	Logs and model training pipeline	Models improve from observed results.
8	GPU inference service	Batch NN and optional local LLM	AI decisions and speech use idle RTX GPU.
9	Economy director ML	Price/supply prediction and gathering allocation	AH feels active and stable.
10	Distributed workers	Networked AI services for scaling	More bots supported without one machine bottlenecking everything.

14. Safe BotBridge Command Examples

The AI service should communicate through a restricted API. Commands should be validated by the server before applying changes.

- PromoteBot(botId, reason, maxDurationSeconds)
- DemoteBot(botId, targetMode)
- AssignAbstractProfile(botId, profileId, durationMinutes)
- ApplyAbstractProgress(botId, xp, money, itemRollProfile, questProgress)
- QueueBotForDungeon(botId, dungeonId, role, skillTier)
- RequestAuctionPost(botId, itemId, quantity, price, duration)
- SetBotPersonality(botId, personalityVector, skillTier)
- GenerateBotChat(botId, context, style, maxTokens)

15. Risks and Guardrails

- CPU meltdown from too many full bots: solve with LOD, throttling, and abstract simulation.
- Economy flooding: solve with quotas, scarcity targets, item caps, gold faucet limits, and rollback metrics.
- Unfair PvP bots: cap reaction time and APM; add human-like mistakes even for Pro bots.
- AI corruption of game state: never allow direct DB writes from AI workers.
- Creepy or spammy bot chat: rate-limit LLM usage; use templates by default; add profanity and safety filters.
- Visible teleporting/immersion breaks: only materialize bots outside player view or with plausible travel delays.
- Model regressions: deploy versioned models and maintain rollback.

16. Recommended Next Build Step

Start with the non-neural foundation that makes the neural system useful later:

1. Add BotSimMode to each bot: FULL, LIGHT, ABSTRACT, OFFLINE.
2. Add distance/visibility/player-relevance checks to choose the mode.
3. Throttle updates based on mode.
4. Add abstract progress formulas for leveling, travel, gathering, and simple questing.
5. Log metrics before and after the change.
6. Only then add NN scoring for promotion/demotion, movement style, and skill/personality tuning.

17. Continuous Bot Training and Improvement Loop

The bot system should not only run bots; it should observe them, detect weaknesses, and improve the next model or code patch. This should be treated as a closed loop: play, collect data, analyze, train, validate, deploy, and monitor.

- Online runtime should focus on safe telemetry and lightweight scoring. Heavy learning should happen offline or in a separate training environment so it does not destabilize the live worldserver.

- Every bot action should be attributable to a model version, ruleset version, behavior-tree version, skill tier, and personality profile. This makes regressions traceable.
- Training should improve high-level decisions, movement style, route selection, combat difficulty tuning, role behavior, economy behavior, and stuck/deadlock avoidance.
- The live server should always keep deterministic validation. Models can suggest better choices, but the server remains the authority for legal movement, combat, quest, loot, auction, and inventory changes.

Play / simulate -> log telemetry -> detect hiccups -> label failures -> train/refine model -> offline validation -> canary deploy -> rollback if worse

18. Telemetry: What to Record From Bot Gameplay

To let bots get better over time, the server needs structured logs rather than only normal text logs. The goal is to record enough information to detect bottlenecks, repeated loops, bad routes, weak combat decisions, and economy side effects.

- Pathing: origin, destination, selected route, distance traveled, road usage percentage, stuck events, terrain retries, jump/facing events, time spent moving, and whether a player could see the bot.
- Combat: opponent type, level difference, class/spec, rotation choices, reaction delays, interrupts, dispels, missed opportunities, potion usage, death reason, flee success, and time-to-kill.
- PvP: opener quality, target choice, cooldown timing, line-of-sight usage, crowd-control chain, reaction time, APM, chase/retreat decisions, win/loss, and whether the bot felt unfair or too slow.
- Questing/leveling: quest chosen, travel time, objective completion rate, drop-rate wait time, failed turn-ins, repeated objective loops, and XP per minute.
- Economy: materials gathered, auctions posted, sales, expired listings, market under/oversupply, price volatility, raw-gold generation, and item flooding risk.
- Deadlocks: same path repeated, same target repeatedly unreachable, repeated vendor/mail/AH failures, repeated death loop, repeated quest objective failure, or bot idling with no progress.

19. Road-Aware Travel and Safe Route Learning

Bots should learn and prefer roads for long-distance travel because roads are usually safer, more believable, and less likely to have creature spawns directly on top of them. This should be implemented as a route-planning layer above the existing pathfinder, not as raw neural movement.

- Create a road graph: extract road-like paths from map data, manually marked waypoints, player travel traces, or bot travel telemetry. Nodes are road junctions, towns, bridges, zone entrances, flight paths, graveyards, and common quest hubs.
- Use road preference for medium/long travel: when a bot needs to travel between areas, route to the nearest safe road node, follow roads through the zone, then leave the road near the destination.
- Score route safety: prefer roads with low creature density, low death rate, low stuck rate, low elite density, and low hostile faction danger for the bot level.
- Let the NN or learned model choose route style: safe-road route, fast direct route, stealthy off-road route, group route, or avoid-contested-zone route depending on personality, level, class, and player visibility.

- Keep micro movement deterministic and legal: once the route chooses road waypoints, normal server pathfinding handles actual legal movement between points. Human-like jitter and jumping are only style overlays.

Long travel plan:

1. current position -> nearest road node
2. road graph route across zone
3. road node near destination -> final pathfinder path
4. if unseen and far from players, use abstract travel progress instead of full movement

20. NN-Enhanced Movement Style Without Breaking Pathing

The NN should make movement look human, not replace the pathfinder. A good split is: deterministic pathfinder chooses legal movement, then the movement-style model chooses believable imperfections.

- Movement style outputs can include pause chance, pause duration, jump/bunny-hop frequency, strafe chance, facing/camera wiggle, path offset, road preference, hesitation near enemies, and follow-distance variance.
- Skill/personality affects movement: a new player may over-jump, pause too long, take safer roads, or backtrack; a pro bot may cut corners, pre-position, strafe in combat, and use cleaner routes.
- Player visibility should raise detail. If a real player is watching, use full legal movement plus humanization. If no player can observe the bot, use light or abstract progress.
- Add guardrails: no impossible zig-zags, no visible teleporting, no excessive bunny hopping in serious RP contexts, no jitter that creates pathing failure, and no movement that bypasses terrain legality.

21. Bot Skill Tiers That Learn Over Time

Skill tiers should be model profiles, not just hardcoded reaction times. Each tier can share the same behavior tree while using different decision weights, reaction windows, mistake rates, and combat priorities.

- Roleplay/Flavor: variable skill, more social behavior, wandering, heavy town/road usage, chatty behavior, and immersion-focused world population.
- Pro: high but still human-capped reaction, strong cooldown timing, LOS, CC chains, target swaps, fake human mistakes, and rare challenge-bot use.
- Hard: faster but capped reaction, better cooldowns, smarter fleeing/focus fire, efficient road usage, better positioning, and fewer stuck events.
- Normal: average reaction, basic rotation, some interrupts, believable pauses/jumps, safe routes, and default dungeon/leveling filler behavior.
- Beginner: slow reaction, low APM, misses interrupts, bad cooldown timing, poor target choice, pauses/backtracks, overuses roads, and works best for low-level world flavor.

Over time, telemetry can update the profiles: if Hard bots still die too often, train better target selection; if Pro bots feel unfair, increase mistake rate or reaction delay; if Beginner bots get deadlocked, patch route logic without making them mechanically strong.

22. Reinforcement Learning Workflow

Reinforcement learning can help, but it should be used carefully. The safest first use is offline training against recorded or simulated environments, then deploying small policy models for high-level choices. Avoid letting a live RL agent freely experiment on the production server.

- Define rewards by role: leveling rewards XP/hour and quest completion; dungeon rewards survival, threat control, healing coverage, and group success; PvP rewards fair challenge, positioning, and win/loss balance; economy rewards healthy supply and stable prices.
- Define penalties: stuck loops, repeated deaths, excessive CPU cost, visible teleporting, market flooding, player frustration signals, too-perfect reactions, and exploit-like movement.
- Train offline from logs first. Use imitation learning from good bot/human traces where possible, then use RL in a sandbox simulation to improve selected decisions.
- Deploy models with versioning and canaries: test on a small bot percentage, compare metrics, and roll back automatically if stuck rate, death loops, CPU cost, or economy volatility increases.
- Keep action spaces small: choose goal, route style, combat tactic, dungeon role behavior, or economy assignment. Do not start with raw movement-key RL for thousands of bots.

State -> policy model -> high-level action -> deterministic server validation -> result -> telemetry -> reward/penalty -> offline training update

23. Detecting Redundant Loops, Deadlocks, and Code Patch Targets

A major benefit of telemetry is finding places where bots repeatedly waste CPU or fail in the same pattern. These should usually become code fixes, route fixes, or behavior-tree fixes before becoming bigger neural models.

- Loop detection: hash recent bot states such as map, zone, position bucket, target, quest objective, action, and failure reason. If the same hash repeats too often with no progress, mark a loop.
- Stuck detection: track low movement delta despite movement commands, repeated path recalculation, repeated unreachable target, or too many position corrections.
- Quest deadlock detection: repeated objective attempt with no credit, repeated turn-in failure, missing item, bad NPC target, or quest objective outside accessible route graph.
- Economy deadlock detection: repeated unsold auctions, bot farming items with no demand, material oversupply, or gold generation above quota.
- Patch target reports: produce daily/weekly ranked lists of top stuck locations, top failed quests, top bad routes, top combat failure states, and top CPU-heavy repeated actions.

Example deadlock report fields:

bot_id, model_version, behavior_version, map, zone, position_bucket, objective, repeated_action, failure_count, elapsed_time, suggested_fix

24. Updated MVP Roadmap Additions

- Add structured bot telemetry tables or CSV logs for pathing, combat, questing, economy, dungeon, PvP, and deadlock events.

- Build a route graph for roads and safe travel nodes; start manually or semi-automatically, then improve from telemetry.
- Add route-choice scoring: direct path vs road-safe path vs abstract travel vs teleport/materialize when unobserved.
- Add movement style profiles controlled by skill/personality: jumps, pauses, mouse-like facing, strafing, path offset, and human mistakes.
- Add deadlock/loop detector and daily reports so repeated bad behavior becomes a code or data patch target.
- Create skill-tier model profiles: Beginner, Normal, Hard, Pro, and Roleplay/Flavor.
- Start with offline imitation/statistical learning from logs; later add reinforcement learning in a sandbox, then canary deploy versioned models.
- Use the GPU for training/inference batches and LLM speech, while the worldserver remains authoritative for legal actions and state changes.